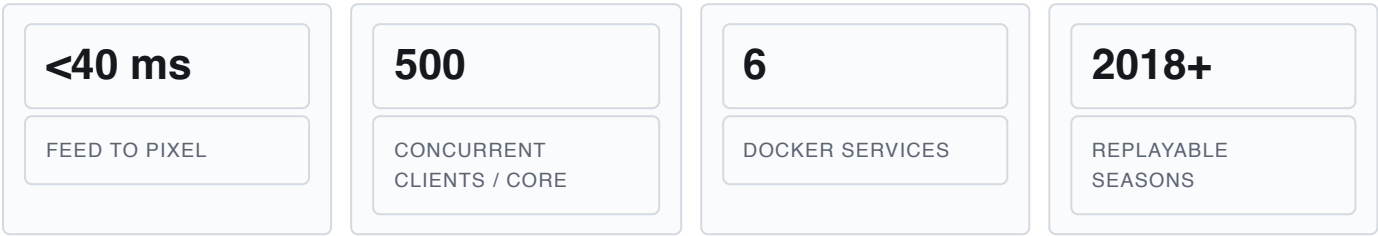


F1 Live Timing & Telemetry

Self-hostable real-time Formula 1 dashboard

.NET 9 · Redis · React · Docker



This report specifies the complete technology stack, runtime architecture, data flow and the engineering decisions that deliver the system's three non-negotiable properties: **speed** (sub-300 ms from data feed to rendered pixel), **efficiency** (hundreds of concurrent viewers on modest hardware) and above all **reliability** (correct state after every disconnect, no silent data loss, honest degradation when the upstream feed is unavailable).

It is a companion to the specification set in docs/00 – docs/14 . Where this report and those documents disagree on implementation language or concurrency primitives, this report wins; where they describe the F1 wire protocol, field names, the merge algorithm or track-map mathematics, those documents remain authoritative.

Contents

1	System overview	3
2	Technology stack	4
3	Runtime architecture	6
4	The live data path	8
5	Performance engineering	9
6	Reliability engineering	11
7	Docker topology	13
8	Replay subsystem	14
9	Delivery plan and risks	15

Project: f1-dash · **Status:** pre-implementation, specification complete
Backend: .NET 9 · **Frontend:** React 19 + Vite · **Runtime:** Docker Compose
Licence: AGPL-3.0 recommended

This project is unofficial and is not associated in any way with the Formula 1 companies. F1, FORMULA ONE, FORMULA 1, FIA FORMULA ONE WORLD CHAMPIONSHIP, GRAND PRIX and related marks are trade marks of Formula One Licensing B.V.

1 System overview

The application streams the same live timing data that powers Formula 1's official broadcast graphics into a browser: a timing tower, a live track map, telemetry traces, race control messages and weather — plus a full replay of any session since 2018, and a user-configurable broadcast delay so the dashboard stays in sync with a TV picture that lags the data.

1.1 The three constraints that shape every decision

Constraint	Consequence for the architecture
The feed is delta-based	F1 sends no periodic snapshots. After one initial state dump, every message is a partial patch — often a single nested field. The server must therefore hold authoritative accumulated state, new clients need a full snapshot on connect, and a generic recursive merge becomes the single most important piece of code in the project.
Arrays arrive as objects	What is logically an array is sent as an object keyed by stringified index — <code>{"Sectors": {"0":..., "2":...}}</code> means "sector 0 and 2 changed, 1 unchanged". A naive object replacement silently corrupts state. The merge must treat numeric-string keys as sparse array indices.
Sessions run ~24 weekends a year	The system must be fully developable and demonstrable from recorded data. A simulator that replays archived sessions in the exact live wire format is infrastructure, not a nicety — it is built before the live client and everything is verified against it.

1.2 Component responsibilities

Service	Runtime	Scale	Owns
ingest	.NET 9 Worker	exactly 1	The single SignalR connection; decompression, normalisation and the authoritative merged state; raw archival recording; session boundary detection. Speaks no HTTP to browsers.
redis	Redis 7	1	State checkpoint, the ordered delta stream, pub/sub wake-up, the ingest singleton lease. State is disposable — no AOF required.
realtime	.NET 9 / Kestrel	1–N	Browser WebSocket fanout; snapshot-plus-delta handshake; cursor resumption; backpressure; the replay engines.
api	.NET 9 / Kestrel	1–N	Stateless REST: schedule, session metadata, results, track geometry, standings, replay bundle metadata. Everything cacheable.
web	React 19 SPA	1–N	Rendering, the client-side delay buffer, position interpolation. Never talks to an F1 endpoint directly.
precompute	Python 3.12	on demand	FastF1-derived lap tables, per-lap telemetry and replay bundles. An offline batch job; never in a request path.
simulator	.NET 9	dev only	Replays a recorded session over WebSocket in the exact live wire format, so ingest cannot tell the difference.

WHY REDIS SITS IN THE MIDDLE

Ingest must be a singleton — two SignalR connections would double-count every delta and risk rate-limiting by F1. Realtime must be replicable — it holds hundreds of browser sockets. Those two requirements are incompatible in one process. Redis Streams give an ordered, ID-addressed, replayable delta log for free, which is also exactly what a reconnecting client needs to resume from without re-downloading a full snapshot.

2 Technology stack

2.1 Backend

Technology	Version	Role and justification
.NET	9.0 LTS	Runtime for ingest, realtime and api. Chosen for <code>System.Threading.Channels</code> (which maps 1:1 onto the bounded per-client queue the fanout design requires), source-generated JSON, first-class async I/O, and a single toolchain across three services.
ASP.NET Core / Kestrel	9.0	HTTP and WebSocket host for realtime and api. Minimal APIs — no MVC, no controllers; these services expose a handful of endpoints each.
System.Text.Json	9.0	Serialisation, with a source-generated <code>JsonSerializerContext</code> : reflection-free, AOT-safe, and measurably the fastest managed JSON path. <code>JsonNode</code> is the storage form for accumulated state.
System.IO.Compression	built-in	<code>DeflateStream</code> for the <code>.z</code> topics. .NET's <code>DeflateStream</code> is <i>raw</i> deflate with no zlib header — precisely what the feed sends. (Reaching for <code>ZLibStream</code> instead is a classic first-day bug.)
StackExchange.Redis	2.8+	Redis client. Note it multiplexes and does not support blocking reads, so the delta reader uses pub/sub wake-up plus non-blocking <code>XRANGE</code> rather than <code>XREAD BLOCK</code> .
Redis	7 (alpine)	Streams for the delta log, a hash for the state checkpoint, pub/sub for wake-up, <code>SET NX PX</code> for the ingest singleton lease.
ClientWebSocket	built-in	The F1 SignalR client. See the trap in §2.4 — the managed SignalR client cannot be used.
prometheus-net	8.x	Metrics at <code>/metrics</code> on every service.
Serilog	4.x	Structured JSON logging to stdout, with once-per-key deduplication for unknown-field warnings.
xUnit + FluentAssertions	current	Unit, fixture-parity and integration tests.

2.2 Frontend

Technology	Version	Role and justification
React	19	UI, with the React Compiler enabled.
Vite	6	Build and dev server. A single-page app, not Next.js: this is a 100% client-side realtime application, so server rendering buys nothing while an App Router adds a layer that fights the WebSocket and the animation-frame loop.
TypeScript	5.7 strict	Types for the F1 state model are generated from the C# records , and CI fails if the committed output is stale.
RTK Query	2.x	All REST data — schedule, standings, track geometry, historical telemetry. Caching, deduplication and invalidation for free.
External store + <code>useSyncExternalStore</code>	React 19	The live F1 state deliberately does not live in Redux. See §5.4.
uPlot	1.6	Telemetry traces. An order of magnitude faster than Recharts or Chart.js at the required update rate, and ~45 KB.

Raw SVG	—	The track map. No charting or mapping library: the geometry is already in the same native coordinate system as the live car positions, so no projection, fitting or scaling is needed.
Tailwind CSS	4	Styling, with design tokens as CSS custom properties for the light and dark themes.
Vitest + Testing Library + Playwright	current	Unit, component and end-to-end tests, the latter driven by the simulator.

2.3 Data sources

Source	Auth	Used for
F1 SignalR live feed <code>livetiming.formula1.com/signalr</code>	none	The live session stream — 17 topics including timing, positions, telemetry, race control and weather. Legacy ASP.NET SignalR 1.5 protocol.
F1 static archive <code>livetiming.formula1.com/static</code>	none	Every session since 2018 as line-delimited streams. The backbone of replay and of every test fixture.
MultiViewer circuits API	none	Track geometry. Critically, its coordinates are in the same native system as the live position feed, so car dots land on the outline with zero fitting.
Jolpica (Ergast successor)	none	Season calendar, results, championship standings. Rate-limited (~4 req/s, 500/hr) so it is refreshed on a daily schedule into a disk cache, never called per request.
FastF1 (Python library)	none	Lap tables, per-lap telemetry and replay bundle precompute. The one reason a Python component exists at all.

NO API KEYS, NO ACCOUNTS, NO VENDOR LOCK-IN

Every upstream is public and unauthenticated. The system has no third-party dependency that can be revoked, priced, or rate-limited out of existence — the only external risk is F1's IP blocking, addressed in §6.5.

2.4 Two decisions that look wrong until explained

.NET'S SIGNALR CLIENT CANNOT CONNECT TO THE F1 FEED

F1 uses *legacy* ASP.NET SignalR 1.5. `Microsoft.AspNetCore.SignalR.Client` implements the SignalR **Core** protocol only and will not connect. The old `Microsoft.AspNet.SignalR.Client` package is in maintenance mode and does not expose the control needed over exact case-sensitive request headers and the negotiate cookie — both of which the feed rejects if they are wrong (HTTP 500 on wrong header case, 400 on a missing one).

The client is therefore a raw `ClientWebSocket` plus a hand-written negotiate and framing layer: roughly 200 lines, and the same work in any language. **This is validated first, in Phase 4, before the rest of the backend commits to .NET.**

PYTHON IS KEPT, BUT CONFINED

FastF1 has no equivalent in .NET or Go, and reimplementing its lap and telemetry parsing would cost roughly a week for no user-visible gain. It is retained as a batch worker that writes bundle files to disk. It never serves a request, so its performance is irrelevant, and if it fails the live dashboard is unaffected.

3 Runtime architecture

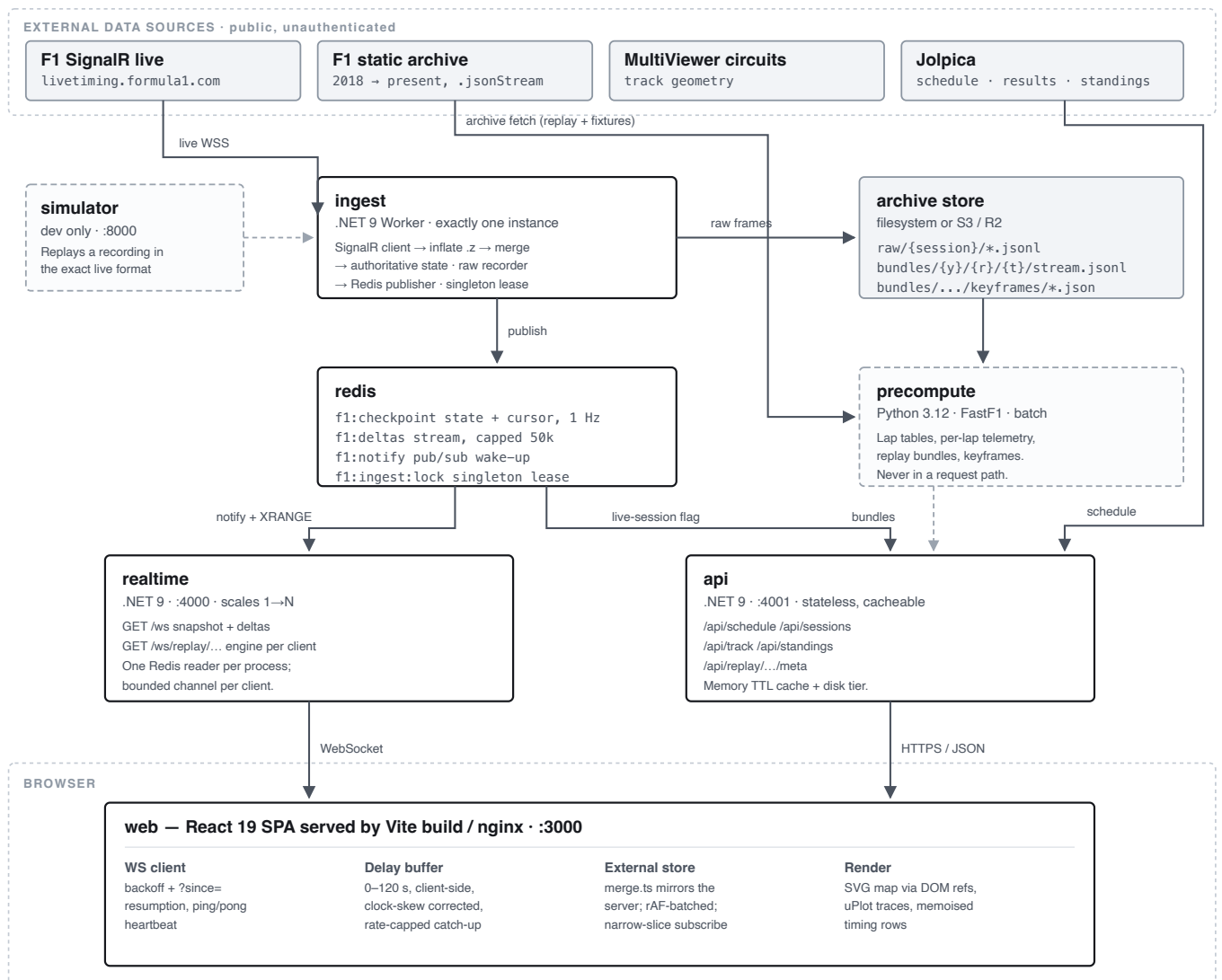


Figure 1 — System architecture. Solid lines carry live data; dashed lines are development-only or batch paths. Ingest is the only component that speaks to the live F1 feed, and the only one that must never be scaled beyond one instance.

3.1 Process boundaries, stated precisely

- ingest** owns the SignalR connection and the authoritative state. It emits normalised deltas — `.z` topics already decompressed, canonical field names preserved. It writes raw untouched wire frames to the archive before parsing them. It speaks no HTTP to browsers.
- realtime** accepts browser sockets, sends a snapshot then deltas, serves replay streams, and drops clients that cannot keep up. It never contacts an F1 endpoint.
- api** is stateless and every response is cacheable. It performs the track-map coordinate transform server-side so the client renders with no mathematics of its own.
- web** never talks to any F1 endpoint directly, and holds the delay buffer.

4 The live data path

CUMULATIVE LATENCY

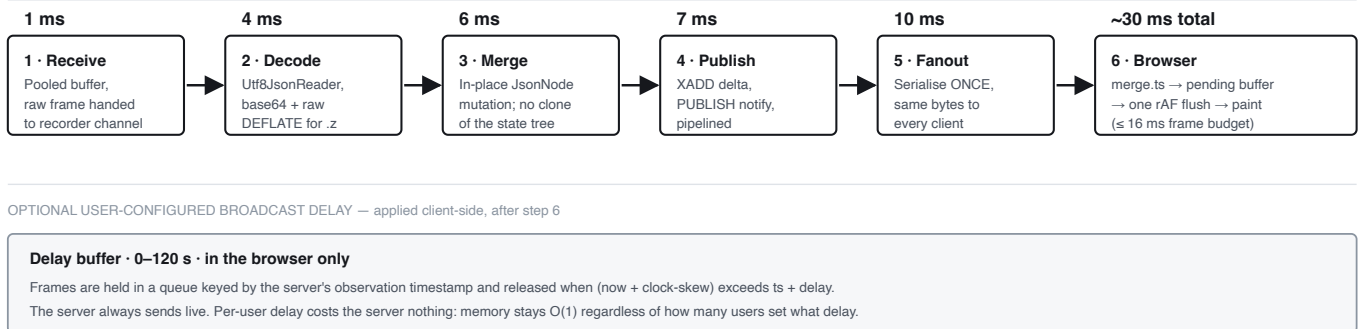


Figure 2 — Latency budget from the F1 feed to a rendered pixel. The stated requirement is under 300 ms; the design budgets roughly 30 ms, leaving an order of magnitude of headroom for network variance.

4.1 Connection handshake

The order of the two reads in the handshake is not arbitrary and is a genuine correctness requirement:

1. The client opens `GET /ws`, optionally with `?since=<streamId>`.
2. The server reads the checkpoint **hash** — snapshot and its cursor together, in one atomic read. Reading them separately allows a delta to slip between the two and be lost forever.
3. It sends `{"type":"snapshot", "data":..., "streamId":..., "serverTime":...}`, using the pre-serialised, pre-gzipped buffer shared by every connecting client.
4. It replays deltas from that cursor to the present with `XRANGE`, then joins the live broadcast. Replaying a delta twice is harmless — merging the same value is idempotent — whereas missing one is not. The design errs deliberately toward duplication.
5. The client computes `skew = serverTime - Date.now()` and re-estimates it every 30 s from the minimum-RTT ping sample, NTP-style.

5 Performance engineering

Five decisions carry essentially all of the system's efficiency. Each is stated with the naive alternative it replaces, because the naive version is what a straightforward reading of the specification produces.

5.1 Snapshot checkpointing, not per-delta snapshot writes

Naive	This design
Write the entire accumulated state to Redis on <i>every</i> delta. With a 1–2 MB state at 10 deltas per second this is 10–20 MB/s of Redis traffic and serialisation — by a wide margin the largest cost in the system, and pure waste.	Write a checkpoint once per second , storing the snapshot and the stream ID it corresponds to atomically in one hash. A connecting client gets that snapshot plus every delta since its cursor. State is byte-identical because the merge is idempotent, at roughly 1/10th the traffic.

5.2 Compress the snapshot once; never compress the deltas

The two payload classes have opposite characteristics and deserve opposite treatment:

Payload	Size	Frequency	Treatment
Snapshot	MB	once per connect	Serialised and gzipped once per checkpoint , cached in memory, and the identical compressed buffer handed to every connecting client. A grandstand of users connecting at lights-out costs one compression, not thousands.
Delta	hundreds of bytes	~10 / s	Sent uncompressed . Per-message deflate would give every client its own compression context and destroy the "serialise once, send the same bytes to everyone" property that makes fanout nearly free. The bandwidth saved is negligible; the CPU cost is per-client and linear.

5.3 One Redis reader per process, and one serialisation per delta

A reader per client would multiply Redis load by the client count and serialise the same message once per client. Instead a single hosted `DeltaBroadcaster` reads, serialises once into a `ReadOnlyMemory<byte>`, and offers that same buffer to every connection. Client count then affects only the number of socket writes — the cost that is genuinely unavoidable.

Each client holds a `Channel.CreateBounded(500)` — roughly two minutes of deltas. A client whose channel fills is closed with code 1013 and reconnects with a fresh snapshot. **A slow client is dropped, never accommodated:** one stalled browser tab must not be able to back up the shared reader for everyone else.

5.4 The live state does not live in Redux

The frontend uses RTK Query for all REST data, where it is the right tool. The live F1 state is different in kind: roughly 10 updates per second against a deeply nested object. A Redux dispatch per delta would re-render the tree and drop frames on mid-range hardware.

- State is a module-level object **outside React**, exposed through `useSyncExternalStore`.
- Incoming deltas accumulate in a pending buffer and flush **once per animation frame**, not once per message. During bursts this alone removes roughly two thirds of the re-renders.
- Components subscribe to the **narrowest possible slice**. A timing row for car 44 re-renders only when car 44's timing changes — never on the whole state object.
- The track map bypasses React entirely for motion: an animation-frame loop linearly interpolates between the last two known positions and writes directly to DOM refs. Setting React state 60 times a second for 20 cars would melt the main thread.

5.5 Share the parsed replay stream; keep only the cursor per client

A two-hour race is roughly 500,000 stream entries. Parsing that per viewer is fatal at the 50-concurrent-replay target. The stream is parsed once into an immutable array, reference-counted with a sliding expiration; each engine holds an index into it plus its own accumulated state of 1–2 MB. Fifty people watching the same race cost one copy of the stream, not fifty.

5.6 Allocation discipline on the hot path

Technique	Effect
<code>ArrayPool<byte></code> for receive and inflate buffers	No large-object-heap churn during a race
<code>Utf8JsonReader</code> over spans	No intermediate <code>string</code> for the envelope
Source-generated serialisation contexts	No reflection, no per-call metadata lookup, AOT-safe
<code>ReadOnlyMemory<byte></code> outbound frames	Zero per-client copies during fanout
Server GC on <code>realtime</code> , workstation GC on <code>ingest</code>	Matches each service's actual concurrency shape
<code>Position</code> and <code>CarData</code> keep the latest	Flat memory across a full race; history is the client's or the bundle's job batch only

None of this is premature optimisation: together these are the difference between roughly 500 concurrent clients on one core and roughly 120.

5.7 Verifiable performance targets

Target	Threshold	How it is verified
Feed to rendered pixel	< 300 ms	Timestamp comparison against the simulator
Concurrent WS clients per process	500	Load test, recorded race at 1×, CPU < 50% of one core
p99 delta latency under load	< 200 ms	500-client load test
Browser long tasks (> 50 ms)	0	Chrome DevTools performance trace
React commits per second, steady state	< 15	React DevTools profiler
Track map frame rate, 20 cars	60 fps	Profiler — zero React commits during steady-state animation
Browser memory growth over 2 h	< 100 MB	Soak test
Backward seek in replay	< 100 ms	With keyframes; < 2 s without
Warm REST response	< 50 ms	Contract tests

6 Reliability engineering

THE GOVERNING PRINCIPLE

Reliability outranks speed here. A dashboard 50 ms faster but showing a stale timing tower after a reconnect is worse than useless, because **the user cannot tell that it is wrong**. Every failure mode below therefore resolves either to correct data or to a visible, honest statement that data is unavailable — never to plausible-looking stale data.

6.1 Ingest is a singleton, enforced at runtime

Two SignalR connections double every delta and risk rate-limiting by F1. Deployment enforces this (`replicas: 1` , `strategy: Recreate`) but deployment mistakes happen, so the process enforces it too: it acquires `SET f1:ingest:lock <id> NX PX 15000` and renews every 5 seconds. Without the lease it runs in standby and publishes nothing. An accidental second instance is harmless; a crashed primary is replaced within 15 seconds.

6.2 Record raw frames before parsing anything

Every frame is appended to the archive byte-identical, **before** any parsing, on a separate channel so it never blocks the hot path. If the parser has a bug the data is not lost and the session can be reprocessed. The same recordings feed the simulator and the regression fixtures. This is the single decision that makes a parsing bug during a live race recoverable rather than terminal — and there is exactly one chance to record any given race.

6.3 Failure modes and defined behaviour

Failure	Defined behaviour
F1 socket drops	Exponential backoff 1 s → 30 s with $\pm 20\%$ jitter, re-negotiating each attempt because the token expires. Accumulated state is deliberately not cleared — the fresh snapshot overwrites it wholesale anyway, and keeping it means the dashboard does not blank out during a brief blip.
Socket alive but silent	No frame of any kind for 30 s is treated as death and forces a reconnect. A dead-but-open socket is common behind proxies and mobile networks, and is invisible without this check.
Redis unavailable	Ingest keeps recording to the archive and buffers; realtime signals clients and retries. Recording never stops — losing data is the only unrecoverable failure in the system.
Browser socket drops	Reconnects with <code>?since=<streamId></code> . If the cursor is still retained, deltas resume with no snapshot. If not, a full snapshot is sent carrying <code>reason: "cursor-expired"</code> so the client knows to reset its store rather than merge onto stale state.
Dead-but-open browser socket	The client pings every 20 s; no pong within 10 s forces a reconnect.
A client stops reading	Its bounded channel fills; it is closed with code 1013 within 30 s and reconnects fresh. Other clients are unaffected.
Delay buffer overflows	The requested delay exceeds the retained window: clamp the delay, drop the oldest frames, and surface a warning rather than silently showing wrong data.
Unknown topic or status code	Rendered neutrally, never dropped, and logged once per distinct key . This is how a feed change is discovered before users report a broken UI — and the feed does change.
Replay capacity exhausted	Beyond <code>MAX_CONCURRENT_REPLAYS</code> (default 50) new replay requests are refused with a clear message rather than degrading everyone's experience.

6.4 Correctness guards that run in CI

Guard	What it prevents
Merge parity	The C# and TypeScript merge implementations run against the same JSON fixtures and must produce identical output. This eliminates the entire class of "the browser shows something different from the server" bugs — the most insidious failure available to a delta-based system, because both sides look internally consistent.
Full-session classification	Replaying a recorded 2024 race through the accumulator must produce a final top ten identical to the official result. One assertion validates parsing, decompression, merging and the state model together. Written in Phase 2 and never allowed to break.
Round-trip recording	Raw frames replayed through the simulator must be byte-identical to the original capture, proving the recording is a faithful substitute for the live feed.
Memory flatness	A full recorded race must not grow the ingest heap, catching accidental accumulation of position or telemetry history.
Generated-type freshness	TypeScript types are generated from the C# records; a stale committed output fails the build, so the two sides cannot silently diverge.

6.5 The one genuine external risk

F1 BLOCKS DATACENTRE IP RANGES

Formula 1 applies IP blocking against known cloud provider ranges, so an ingest process running on a typical VPS may be unable to connect. Mitigations in order of preference:

1. Run **only** ingest on a residential or non-datacentre connection — a home server or a small Raspberry Pi — writing to Redis in the cloud. Everything else deploys normally.
2. Route ingest's outbound traffic through a proxy via `F1_HTTP_PROXY`.
3. Fall back to replay-only mode.

The architecture is built so that the third option is not a catastrophe: `api` and `realtime` touch no F1 endpoint at request time, so schedule, standings, results and the entire replay experience remain fully functional. The dashboard says "*no live session*" rather than showing an error — which is also, correctly, what it says for the ~340 days a year when no session is running.

6.6 Observability

Every service exposes Prometheus metrics at `/metrics` and logs structured JSON to stdout. The signals that matter most:

Metric	Why it is the one to watch
<code>f1_ingest_connected</code>	The single most important boolean in the system
<code>f1_ingest_frames_total{topic}</code>	A topic going silent is the earliest signal of a feed change
<code>f1_ingest_lease_held</code>	Confirms the singleton invariant actually holds
<code>f1_realtime_delta_lag_seconds</code>	End-to-end freshness, the user-visible quality measure
<code>f1_realtime_dropped_clients_total{reason}</code>	Separates backpressure drops from genuine errors
<code>f1_replay_engines_active</code>	Headroom against the concurrency cap

7 Docker topology

Everything runs in Docker, including development. The only tools required on the host machine are Docker and an editor — no .NET SDK, no Python, no Node installation is needed to build or run the stack. `docker compose up` produces a working system.

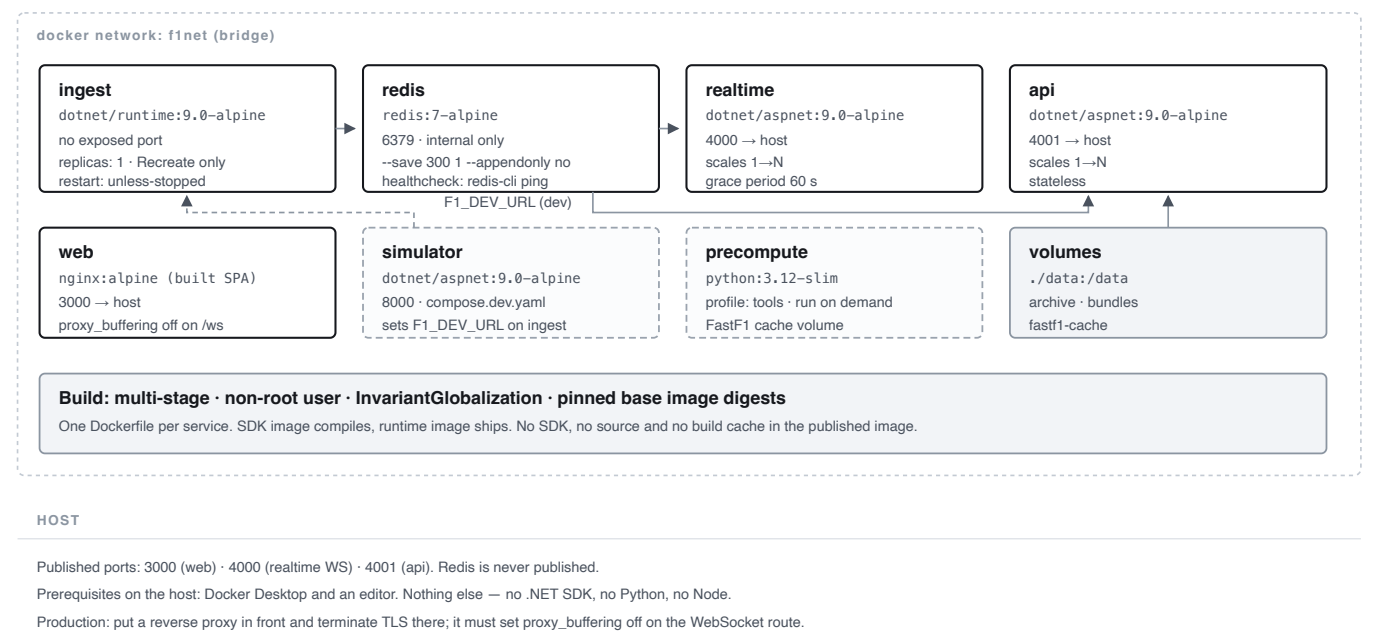


Figure 3 — Container topology. Dashed containers are development or on-demand tooling and are absent from a production `compose up`.

7.1 Image and build discipline

- **Multi-stage builds.** The SDK image compiles; the runtime image ships. No SDK, source or build cache reaches the published image.
- **Alpine runtime images** with `InvariantGlobalization=true` — the application has no locale-dependent formatting.
- **Non-root user** in every image.
- **Pinned base image digests**, not floating tags, so a rebuild six months from now produces the same image.
- **Health checks** on every service, with `depends_on: condition: service_healthy` so start-up ordering is deterministic.
- **ingest uses strategy: Recreate, never rolling update** — two instances must never overlap, not even for the seconds of a deployment.
- Optionally `PublishAot` for ingest: fast start and a small footprint, viable because serialisation is source-generated and needs no reflection.

7.2 Development workflow

Command	Result
<code>docker compose up</code>	Production-shaped stack: redis, ingest, realtime, api, web
<code>docker compose -f compose.yaml -f compose.dev.yaml up</code>	Adds the simulator and sets <code>F1_DEV_URL</code> , so the whole stack runs from a recorded race with no live session and no network access to F1
<code>docker compose run --rm precompute --year 2024 --round 1</code>	Builds a replay bundle for one session
<code>docker compose run --rm tests</code>	The full test suite in a container, identical to CI

The second command is the one used for the overwhelming majority of development, since sessions run on roughly 24 weekends a year and the system must be fully developable on the other 340 days.

8 Replay subsystem

Replay shares the delay feature's machinery and reuses the entire live rendering path — the same components, driven by a different source. Two modes exist.

Mode	Fidelity	How it works
Raw stream replay (preferred)	Exact	Archived <code>.jsonStream</code> files are replayed through the same accumulator the live feed uses, so rendering is byte-for-byte identical to live. Mini-sectors, timing tower behaviour and flag transitions are all genuine.
Precomputed bundle (fallback)	Reduced	For sessions whose raw archives are missing or malformed, FastF1 samples fixed 500 ms frames. Playback is a trivial time index, but there are no mini-sectors and no exact timing tower behaviour. Fallback only.

8.1 Seeking is a state rebuild

Because state is delta-accumulated, you cannot jump backwards without replaying from a known point — there is no such thing as reading the state at time t directly. A naive backward seek on a two-hour race replays hundreds of thousands of entries and takes roughly two seconds, which makes scrubbing unusable.

Keyframes solve this. A full state snapshot is precomputed every 60 seconds of session time. A seek then loads the nearest preceding keyframe and replays at most 60 seconds of deltas, bringing backward seek under 100 ms. Built in Phase 12, deliberately after correctness — the correct-but-slow version ships first and is what the fast version is validated against.

8.2 Bundle contents

File	Contents
<code>meta.json</code>	Session info, driver list, duration, lap markers, and the event markers that drive the scrub bar
<code>stream.jsonl</code>	All topics merged into one time-ordered delta list
<code>keyframes/{offset}.json</code>	Full state every 60 s of session time
<code>track.json</code>	Cached MultiViewer geometry, already transformed
<code>laps.json</code> , <code>results.json</code>	FastF1 lap table and final classification

Event markers deserve emphasis: they are derived from race control messages and track status transitions during precompute, and they are what a user actually seeks to — the safety car, the red flag, the incident. A scrub bar without them is a scrub bar nobody uses.

8.3 Precompute triggering

1. **On demand** — a user opens an unprocessed session; a background job starts and streams progress over server-sent events. Cold processing takes 30–120 s, so the UI shows **named stages with progress**, not a spinner. A spinner for 90 seconds reads as a broken page.
2. **Bulk CLI** — `docker compose run --rm precompute --year 2024 --all`.
3. **Scheduled** — a background task checks every 30 minutes on race weekends and processes sessions as they finish, so the common case is already warm before anyone asks.

9 Delivery plan and risks

9.1 Phased build order

Phases are defined by capability, not by language, and each ends in a commit with its own acceptance check. No phase begins before the previous one's check passes.

Phase	Deliverable	Effort	Acceptance check
0	Docker scaffold, solution, lint and CI	0.5 d	<code>docker compose up</code> starts Redis; checks pass on an empty repo
1	Real data on disk	1–2 d	A 2024 race downloaded and merged into a time-ordered stream with >100k lines; positions inflate to valid JSON for 20 drivers
2	Merge algorithm	2 d	The final top ten from the fixture matches the official race result exactly. This single check validates the whole parsing and merge layer.
3	Simulator	1–2 d	Frames arrive at a realistic rate in genuine live-feed shape, with <code>.z</code> topics re-compressed so the decompression path is exercised
4	Ingest service (<i>incl. the SignalR spike</i>)	2–3 d	State in Redis holds 20 drivers; the delta stream grows steadily; the live client connects and stays up 10 minutes
5	Realtime service	2 d	Two clients receive a snapshot then identical deltas; killing one does not affect the other
6	Minimal dashboard, end to end	2 d	A live-updating leaderboard in the browser, driven by the simulator
7	Track map	2–3 d	Bahrain, Monaco and Suzuka render correctly; dots stay on the track for a full lap at 60 fps
8–9	Full timing tower and panels	5–6 d	Information content matches the reference; a safety car triggers banner, sector colouring and the race control feed together
10	Broadcast delay	2 d	30 s delay visibly holds the tower back; reducing 60 s → 0 does not freeze the tab beyond 500 ms; ± 5 s clock skew changes observed delay by < 200 ms
11	Telemetry	2–3 d	A fastest-lap speed trace matches FastF1's own plot
12	Replay	5–7 d	A full race replays to a correct final classification; seek settles within budget; 4× drops no deltas
13–14	Schedule, standings, hardening	4–5 d	500-client load test passes; 24 h soak shows < 10% memory growth
15	Live validation	1 weekend	Positions match official timing at ten random moments; gaps within 0.1 s; the session records without gaps

Roughly 6–8 weeks of focused solo work for full parity. A genuinely usable dashboard exists at the end of Phase 9 — about three weeks in.

9.2 If time runs short

Drop in this order: telemetry comparison, then replay (the single largest chunk of work), then standings. **Never cut the merge algorithm's tests, the simulator, or the delay feature** — those three are load-bearing. Cutting the simulator in particular makes every subsequent phase unverifiable outside a race weekend.

9.3 Risk register

Risk	Likelihood	Impact	Mitigation
F1 blocks the ingest IP	High	High	Run ingest on a non-datacentre connection; proxy support built in; the app degrades to a fully functional replay-only mode
The feed's shape changes mid-season	Medium	Medium	Never drop unknown fields; log every unrecognised topic and status once; raw recordings allow reprocessing after a fix
The legacy SignalR handshake resists implementation	Medium	High	Spiked first, in Phase 4 , before the backend commits. Header case and the negotiate cookie are the known failure points and are documented precisely.
Live edge cases absent from archives	High	Medium	Phase 15 reserves a full race weekend; red-flag restarts, mid-session driver changes and feed dropouts are explicitly enumerated as test cases
Frontend performance under burst load	Medium	Medium	Animation-frame batching, narrow-slice subscriptions and DOM-ref map rendering are designed in from Phase 6, not retrofitted; profiler thresholds are acceptance criteria
Replay memory at 50 concurrent viewers	Low	Medium	The parsed stream is shared across engines; only cursor and state are per-client; a hard concurrency cap refuses rather than degrades
Jolpica rate limits	Low	Low	Daily refresh into a disk cache; never called per request; token-bucket limited

9.4 Summary of the argument

The workload is not throughput-bound — 50–200 KB/s and roughly ten deltas per second are modest for any modern runtime. What the system is actually bound by is **correctness under a delta-based protocol** and **fanout efficiency**. The architecture therefore spends its complexity budget in exactly two places: a single, generic, heavily-tested merge implementation shared in behaviour between server and browser, and a fanout path that serialises once and compresses once regardless of how many people are watching.

Everything else — the language choice, the container layout, the caching tiers — follows from those two commitments and could be substituted without changing the shape of the system. That is the intended property: the load-bearing decisions are few, explicit, and verified by tests that fail loudly.

References. Full specification in docs/00 — docs/18 . Wire protocol: docs/03 , docs/04 . Data model: docs/05 . Merge rules: docs/06 . Fanout: docs/07 . Delay and replay: docs/08 . Track map mathematics: docs/10 . .NET mapping: docs/16 . Use cases: docs/15 . Prerequisites: docs/18 .